

Note de Cadrage & Plan de Développement (CC1)

Projet DIYPG — Do It Yourself Privacy Guard (Cryptographie & Blockchain)

Établissement : Université Toulouse III - Paul Sabatier

Diplôme : Licence Informatique (L2 S4)

Équipe : Ethan NAPOLETANO, Dorian BOUSSEKSOU,

Enseignant / Tuteur : M. Vincent DUGAT

Étudiante : Tahar MOKHTARI, Clément BOILAT

1. NOTRE COMPRÉHENSION DU SUJET DIYPG

Ce que nous avons compris du projet : L'objectif principal est de concevoir en C modulaire une application inspirée du logiciel standard *Gnu Privacy Guard (GPG)*. Plutôt que de coder des fonctions isolées, le sujet nous amène à fabriquer un véritable environnement de sécurité qui combine :

- **Confidentialité** : Chiffrement asymétrique RSA de messages et fichiers.
- **Authenticité & Intégrité** : Hachage SHA-256 et signature numérique de documents.
- **Ergonomie** : Un interprète de commandes (Shell CLI) interactif avec annuaire de contacts.
- **Confiance décentralisée** : Une Blockchain simulée faisant office d'autorité de certification des clés publiques.

2. CE QUE NOUS ALLONS PROGRAMMER (SPÉCIFICATIONS DES PHASES)

Pour mener à bien le développement, nous suivons le découpage en 5 phases proposé dans le sujet :

Phase 1 : Socle RSA & Outils de Base

- **1.0 Arithmétique** : Implémentation du crible d'Ératosthène, du test statistique de Rabin, d'Euclide étendu (Bézout) et génération de clés RSA sur 64 bits (`uint64_t`).
- **1.1 - 1.2 Chiffrement & Fichiers** : Chiffrement octet par octet puis lecture/écriture dans des fichiers binaires.
- **1.3 Base64 & Tests** : Conversion binaire ↔ Base64. Analyse expérimentale du dépassement de capacité avec `MAX_PRIME`.

Phase 2 : Chiffrement par Blocs & GMP

- **Chiffrement par blocs** : Regroupement d'octets par blocs de 4 octets (`uint32_t`) avec conversion *little-endian* pour contrer l'analyse fréquentielle.
- **Intégration GMP** : Utilisation de la bibliothèque multiprécision GMP (`mpz_powm`) pour traiter les grandes puissances modulaires sans débordement.
- **Échange de clés** : Transmission de clés publiques au format Base64 entre équipiers.

Phase 3 : Interprète de Commandes (CLI) & Annuaire de Contacts

- **Boucle REPL** : Shell interactif traitant les commandes saisies au clavier par l'utilisateur.
- **Gestion du Trousseau (V1.1)** : Commandes `newkeys`, `rmkeys`, `listkeys`, `crypt`, `uncrypt`, `show`, `save` et `load`.
- **Annuaire Contacts (V2.0)** : Support des identifiants contacts (ex: `idcontact/idclef`), commandes `addcontact`, `modifycontact`, `listcontacts`.
- **Protection** : Sauvegarde chiffrée/protégée de l'annuaire local.

Phase 4 : Signatures Numériques & Hachage SHA-256

- **Empreinte numérique** : Calcul du digest SHA-256 d'un fichier (en clair ou chiffré).
- **Signature RSA** : Chiffrement du digest avec la clé privée de signature de l'émetteur (commandes `sigtext`).
- **Vérification d'authenticité** : Déchiffrement du digest avec la clé publique du signataire et comparaison avec l'empreinte recalculée pour garantir l'intégrité et la paternité (commande `verifysign`).

Phase 5 : Registre d'Authentification des Clés par Blockchain

Simulation d'une autorité de certification centralisée sous forme de chaîne de blocs :

- **Structure de données** : Blocs chaînés par hash SHA-256, timestamp, nonce et liste de transactions.
- **Arbre de Merkle** : Calcul de la racine de Merkle pour valider l'ensemble des transactions du bloc.
- **Proof of Work** : Moteur de minage imposant un nombre de zéros initiaux (`difficulty`).
- **Tokens & Événements** : Comptes crédités (100 tokens user, 1000 institution) et transactions de clés : NCK (nouvelle), RCK (révocation), NSK, RSK.
- **Audit** : Fonction de contrôle d'intégrité globale de la chaîne depuis le bloc Génésis.

3. ARCHITECTURE LOGICIELLE & DÉCOUPAGE MODULAIRE

Afin d'obtenir un code propre, lisible et testable, nous organisons le projet en plusieurs modules C :

Module C (Fichiers)	Rôle & Fonctions Principales	Bibliothèques / Dépendances
<code>rsa_core.c / .h</code>	Génération des clés RSA, Miller-Rabin, Bézout, conversion blocs 4 octets et puissance modulaire GMP.	<code><stdint.h></code> , <code><gmp.h></code>
<code>other_base64.c / .h</code>	Encodage/décodage Base64 pour buffers mémoire et fichiers.	<code><stdlib.h></code>
<code>keyring.c / .h</code>	Gestion du trousseau utilisateur, annuaire des contacts V1/V2, sauvegarde chiffrée sur disque.	<code>rsa_core.h</code> , <code>other_base64.h</code>
<code>sha256_sign.c / .h</code>	Hachage SHA-256, signature d'empreinte et vérification d'authenticité.	<code>sha256.h</code> , <code>rsa_core.h</code>
<code>blockchain.c / .h</code>	Structures Block/Transaction (<code>'bc_rsa.h'</code>), Arbre de Merkle, minage PoW, gestion des tokens et audit.	<code>sha256.h</code>
<code>cli_interpreter.c / .h</code>	Boucle REPL, parsing des commandes utilisateur et dispatch vers les modules.	Tous les modules C
<code>main.c</code>	Point d'entrée du programme principal, initialisation des structures et lancement du Shell.	<code>cli_interpreter.h</code>

4. RÉPARTITION DES TÂCHES ENTRE ÉTUDIANTS

Chaque membre de l'équipe est responsable de sous-systèmes précis afin de travailler en parallèle :

Étudiant	Rôle Majeur	Modules & Tâches Attribuées
Dorian BOUSSEKSOU	RESPONSABLE CLI ANNUAIRE & REPL	• Conception de l'interprète de commandes CLI (<code>'cli_interpreter.c'</code>, <code>'main.c'</code>). • Gestion de l'annuaire utilisateur V1.1 et des contacts V2.0 (<code>'keyring.c'</code>). • Sauvegarde/chargement chiffré du trousseau de clés. • Rédaction du Makefile principal et du fichier README d'utilisation.
Ethan NAPOLETANO	LEAD CŒUR RSA CALCULS GMP	• Implémentation du socle arithmétique RSA (Miller-Rabin, Bézout, Ératosthène). • Découpage par blocs de 4 octets et gestion de l'endianness (<code>'int2char.c'</code>). • Intégration de la bibliothèque GMP pour les calculs multiprécision. • Analyse des limites d'overflow et tests avec la constante <code>'MAX_PRIME'</code>.
Tahar MOKHTARI	SIGNATURES SHA-256 MODULE BASE64	• Implémentation du module de conversion Base64 mémoire et fichier (<code>'other_base64.c'</code>).

5. ÉCHÉANCIER PRÉVISIONNEL & JALONS DE CONTRÔLE CONTINU

Notre planning s'étale de septembre à décembre 2026, structuré autour des 4 évaluations (CC) du projet :

Période	Phase du Projet	Objectifs Techniques & Livrables Intermédiaires	Jalon & Note
Sem. 1 – 2 (Sept. 2026)	Cadrage & Phase 1.0 - 1.2	- Création du dépôt Git, squelette de code et Makefile. - Validation des fonctions RSA basiques (Miller-Rabin, Bézout). - Premiers tests de chiffrement octet par octet sur fichiers.	CC1 (10%) Note de Cadrage
Sem. 3 – 4 (Oct. 2026)	Phase 1.3 & Phase 2 (GMP & Blocs)	- Validation du module Base64 (mémoire et fichiers). - Passage au chiffrement par blocs de 4 octets avec GMP. - Échange de fichiers chiffrés et de clés entre équipiers.	Validation Phase 1 & Phase 2
Sem. 5 – 7 (Fin Oct. / Nov.)	Phase 3 (Interprète CLI & Annuaire)	- Implémentation du Shell interactif REPL. - Gestion de l'annuaire V1.1 et des contacts V2.0. - Chiffrement de la sauvegarde de l'annuaire local.	CC2 (20%) & CC3 (20%) Validation P1, P2 & P3
Sem. 8 – 10 (Novembre)	Phase 4 & Phase 5 (Signatures & Blockchain)	- Intégration SHA-256 et signatures de digests. - Construction de la Blockchain, Arbre de Merkle, minage PoW et tokens.	Intégration Globale
Sem. 11 – 12 (Déc. 2026)	Recette & Démonstration	- Chasse aux fuites mémoire avec Valgrind. - Vérification de l'intégrité globale du système. - Finalisation de la documentation et préparation de la soutenance.	CC4 (50%) Recette Finale

6. RISQUES IDENTIFIÉS & PLAN DE PARADES

Risque Identifié	Impact Potentiel	Stratégie de Parade Étudiante
Dépassement uint64_t	Erreurs de calcul lors de l'exponentiation modulaire en Phase 1.	Tests rigoureux sur la constante MAX_PRIME et bascule contrôlée vers GMP en Phase 2.
Fuites Mémoire (Pointeurs C)	Plantage lors de la manipulation des listes chaînées (Blockchain) ou des objets GMP.	Usage systématique de fonctions free_* dédiées et vérification Valgrind avant chaque rendu.
Erreurs de Compilation GMP	Échec de build sur certaines machines de l'équipe.	Rédaction d'un Makefile robuste incluant explicitement le flag -lgmp et documentation dans le README.
Rupture du chaînage Blockchain	Non-cohérence du Merkle Tree ou des hashes de blocs lors du minage.	Création d'une fonction de vérification d'intégrité autonome testant la chaîne depuis le bloc Génésis.

Engagement de l'Équipe Projet DIYPG :

Nous soussignés confirmons notre organisation de travail, la répartition des modules et notre engagement à respecter cet échéancier pour la réussite du projet en décembre 2026.

Responsable CLI Dorian BOUSSEKSOU ----- Lu et approuvé	Lead Cœur RSA Ethan NAPOLETANO ----- Lu et approuvé	Signatures & Base64 Tahar MOKHTARI ----- Lu et approuvé	Lead Blockchain Clément BOILAT ----- Lu et approuvé
---	--	--	--